KLAi

Optimisation in practice: keep the weights small, then keep the numbers in range

How do we limit memorising in a model with billions of numbers, and why does a very long answer not overflow the computer?

Mathematics for AI and Machine Learning | Session 12 of 18 | Calculus and optimisation

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Fit a curve with a ridge (L2) penalty, choose lambda on validation points, and report the test error once

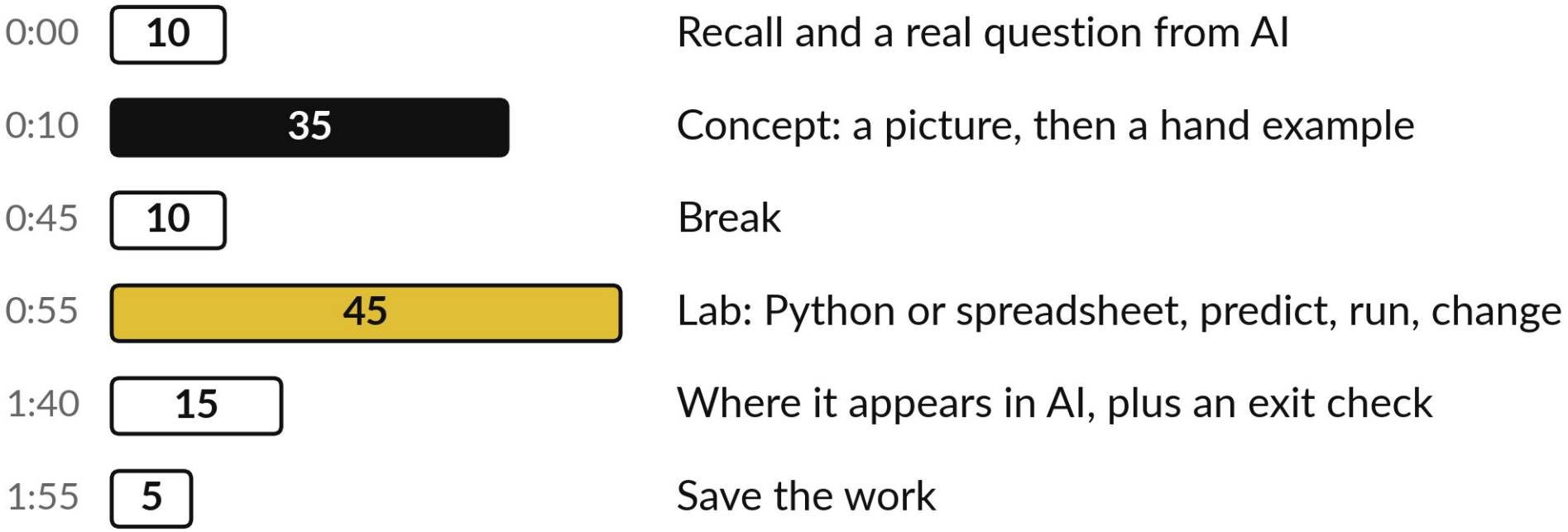


Explain weight decay and a limit on the size of the weights as two faces of the same penalty



Compute a softmax by hand, then compute it and a log-sum-exp without overflow by subtracting the maximum

Your two hours: sensible weights first, then sensible numbers



Part A: a penalty against overfitting. Part B: softmax and arithmetic that stays in range. Then the lab.

Three questions from last time

1. In our tiny network, what is delta(2) and what value did it have?

2. Why does the hidden delta contain a × (1 minus a)?

3. Which values from the forward pass does the backward pass reuse?

Answer aloud first. Then we check together.

Answers to the three questions from Session 11

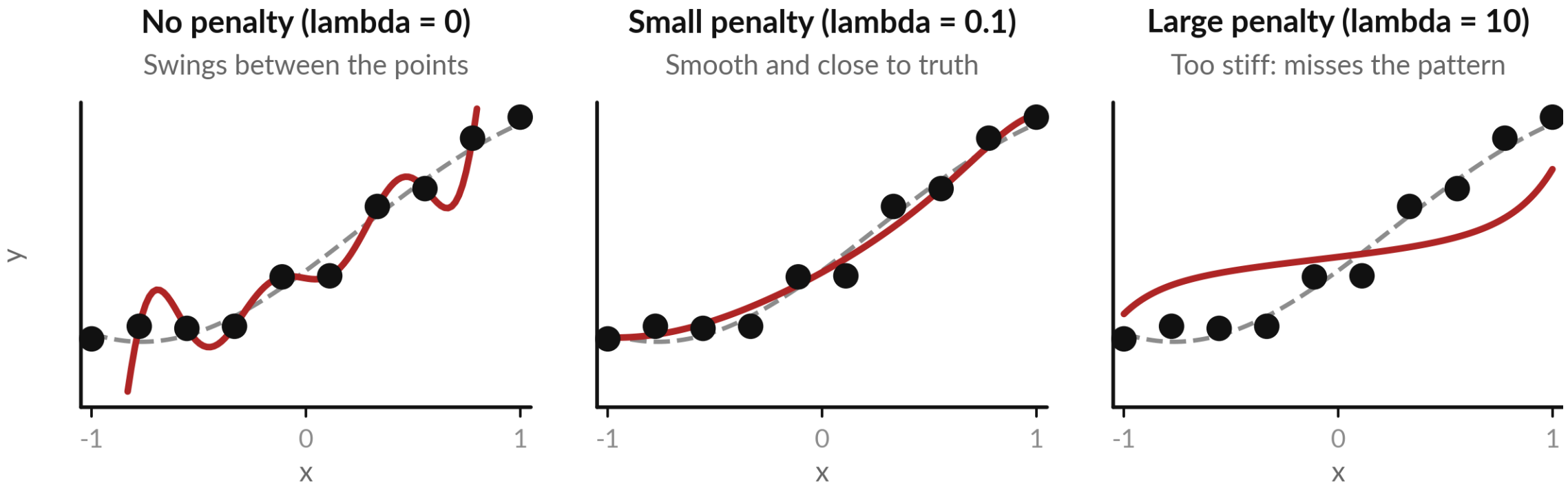
Question	Short answer
In our tiny network, what is delta(2) and what value did it have?	delta(2) is the slope of the loss with respect to the prediction, y-hat - y. It was 0.1733 - 1 = -0.8267.
Why does the hidden delta contain a × (1 minus a)?	a × (1 - a) is the slope of the sigmoid. The chain rule multiplies it into the slope that arrives from the output.
Which values from the forward pass does the backward pass reuse?	The hidden outputs a(1) and the inputs x, which multiply the deltas to give the gradients. They are stored during the forward pass.

Check your answers. If one surprised you, look at Session 11 before the lab.

Words we use today

	Word	What it means
	**Regularisation**	Any change to training that discourages a model from memorising its training data.
	**L2 penalty (ridge) and weight decay**	Lambda times the sum of the squared weights, added to the loss. In plain gradient descent it shrinks every weight a little at each update.
	**Softmax**	Turns a list of scores (logits) into probabilities that are positive and add up to 1.
	**Overflow and underflow**	A result too large or too small for the number format: it becomes inf or 0.
	**Log-sum-exp**	m + ln(sum of e^(z - m)) with m the largest score: the log of a sum of exponentials, computed safely.

A curve with too much freedom chases the noise: that is overfitting



Part A. Dots: 10 training points. Dashed: true curve. Red: the degree 9 fit. Illustrative data.

The L2 (ridge) penalty adds the size of the weights to the loss

Ridge loss (sum of squared errors plus penalty)

$$L(w) = \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^{d} w_j^2$$

Prediction

$$\hat{y} = b + w_1 x + w_2 x^2 + \ \ldots \ + w_9 x^9$$

Symbol	What it means	In our data
lambda	The penalty strength, chosen by us. Larger means smaller weights.	0, 0.001, 0.01, 0.1, 1, 10
sum of w^2	The squared L2 norm of the weights (Session 2). The intercept b is left out.	about 104,000 at lambda = 0
n, d	Number of training points; number of penalised weights.	n = 10, d = 9
First term	The sum of squared errors, SSE (Session 10). Libraries often use the mean: then the same effect needs a lambda n times smaller.	SSE, no division by n

First term: fit the data. Second term: keep the weights small. Lambda sets the balance.

By hand: one weight, three points, and what lambda does to it

lambda	Formula: w = sum(x y) / (sum(x^2) plus lambda)	Weight w	Share of the plain fit
0	28.5 / 14	2.0357	100%
1	28.5 / 15	1.9000	93%
14	28.5 / 28	1.0179	50%
140	28.5 / 154	0.1851	9%

Data (illustrative): x = 1, 2, 3 and y = 2.1, 3.9, 6.2, so sum(x y) = 2.1 + 7.8 + 18.6 = 28.5 and sum(x^2) = 1 + 4 + 9 = 14. Model: y-hat = w x.

Ridge has a closed-form answer: add lambda to the diagonal, then solve

Closed form

$$w = (X^T X + \lambda P)^{-1} X^T y$$

Symbol	What it means	In our data
X	Design matrix: one row per point, columns 1, x, ..., x^9. A line (Session 4) uses two columns.	x = 0.333: 1, 0.333, 0.111, 0.037, ...
X^T X	The square matrix of the normal equation (Session 4).	10 × 10
lambda P	Lambda on the diagonal. P is the identity with first entry 0: no penalty on the intercept.	lambda = 0.1
Badly conditioned	Tiny changes in y or rounding cause huge changes in w (condition number).	about 2 × 10^7 at lambda = 0

In practice a solver is used instead of an explicit inverse. Adding lambda makes the matrix well behaved.

Choose lambda on validation points: among six values, 0.1 gives the lowest validation error

lambda	Sum of the squared weights	Training error (RMSE)	Validation error, 20 points (RMSE)
0	103,994	0.0000	0.6657
0.001	3.3569	0.0890	0.1662
0.01	2.4827	0.0908	0.1636
0.1	1.5473	0.1092	0.1584
1	0.6144	0.2181	0.2039
10	0.1181	0.4549	0.4198

RMSE = root mean squared error. Validation points choose lambda; 20 test points, unused until then, give 0.173 once. A finer grid finds 0.2 slightly lower (0.1569).

Weight decay: every update first shrinks the weights by a small factor

L2 penalty in the update

$$w \leftarrow w - \eta\left(\frac{\partial L}{\partial w} + 2\lambda w\right)$$

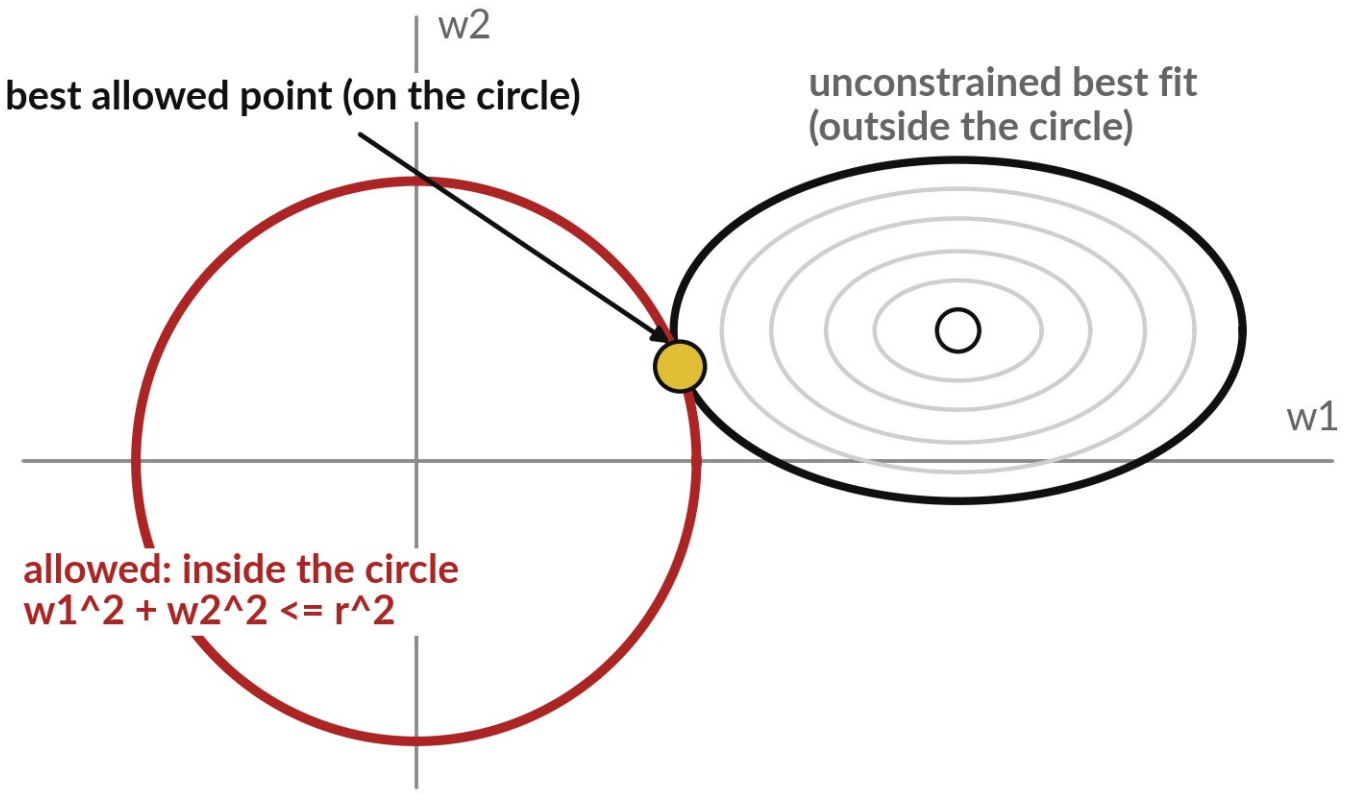
Weight decay form

$$w \leftarrow (1 - 2\eta\lambda)\, w - \eta \frac{\partial L}{\partial w}$$

Symbol	What it means	In our data
eta	The learning rate (Session 10).	eta = 0.1
dL/dw	The slope of the loss without the penalty.	illustrative: 1
lambda	The penalty strength. In plain SGD libraries the weight_decay setting equals 2 lambda.	lambda = 0.5
1 minus 2 eta lambda	The shrink factor applied to w before the gradient step.	1 - 2 × 0.1 × 0.5 = 0.9

Worked: w = 2 gives 0.9 × 2 - 0.1 × 1 = 1.7. AdamW (Loshchilov and Hutter, 2017) shrinks in a separate step.

A penalty and a limit on the weights are two ways to say: keep the weights small



Ridge = the best weights inside a circle. Larger lambda, smaller circle.

The best allowed point is where a loss ring just touches the circle.

On the line w1 + w2 = 2, the smallest w1^2 + w2^2 is 2, at (1, 1).

From sensible weights to sensible numbers: a correct formula can still fail on a computer

Part A kept the weights sensible. Part B keeps the numbers sensible.



A computer stores each number in a fixed number of bits.

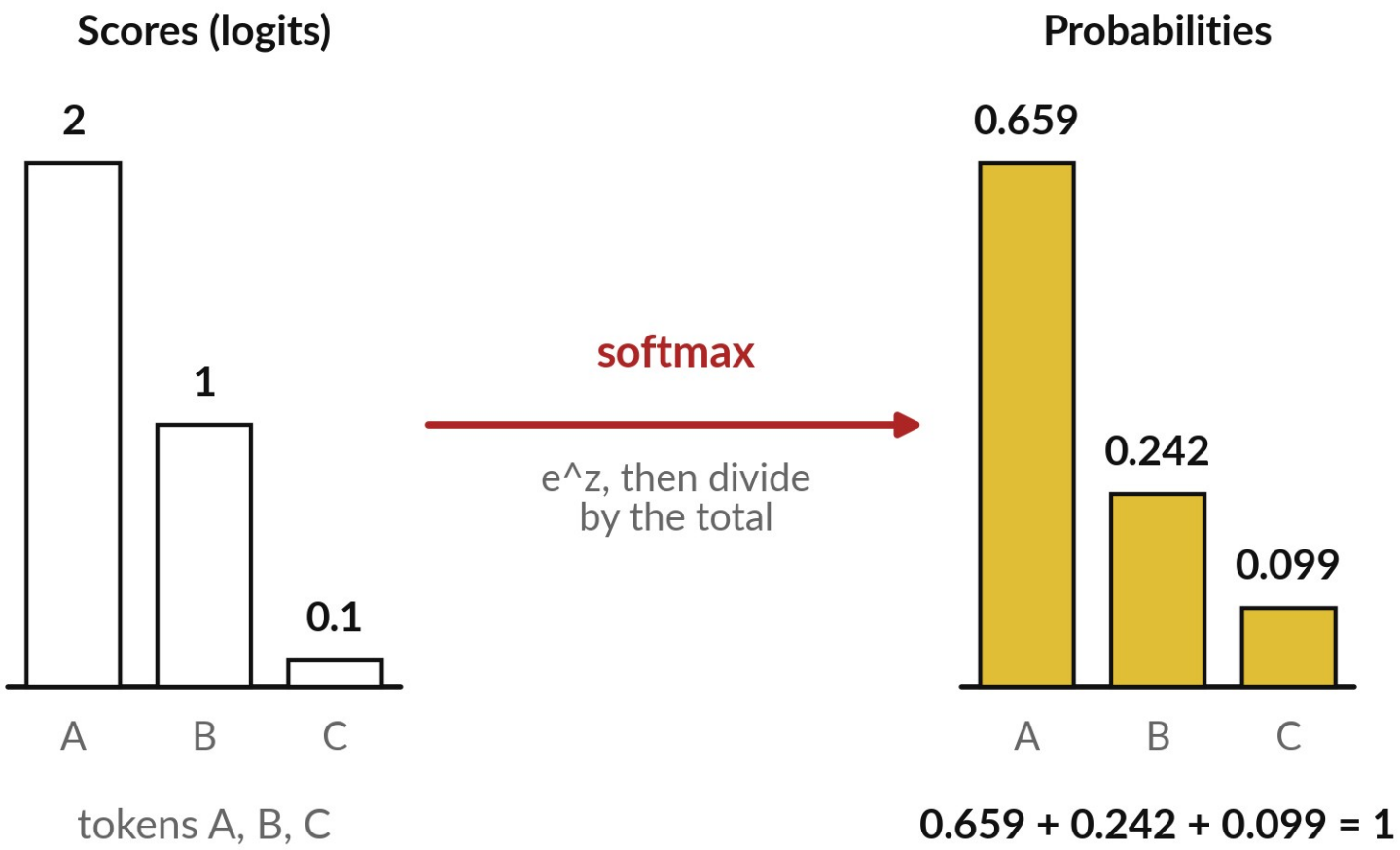


A result too large becomes inf (overflow); too small becomes 0 (underflow).



inf / inf gives nan, short for not a number.

Softmax turns any list of scores into probabilities that add up to 1



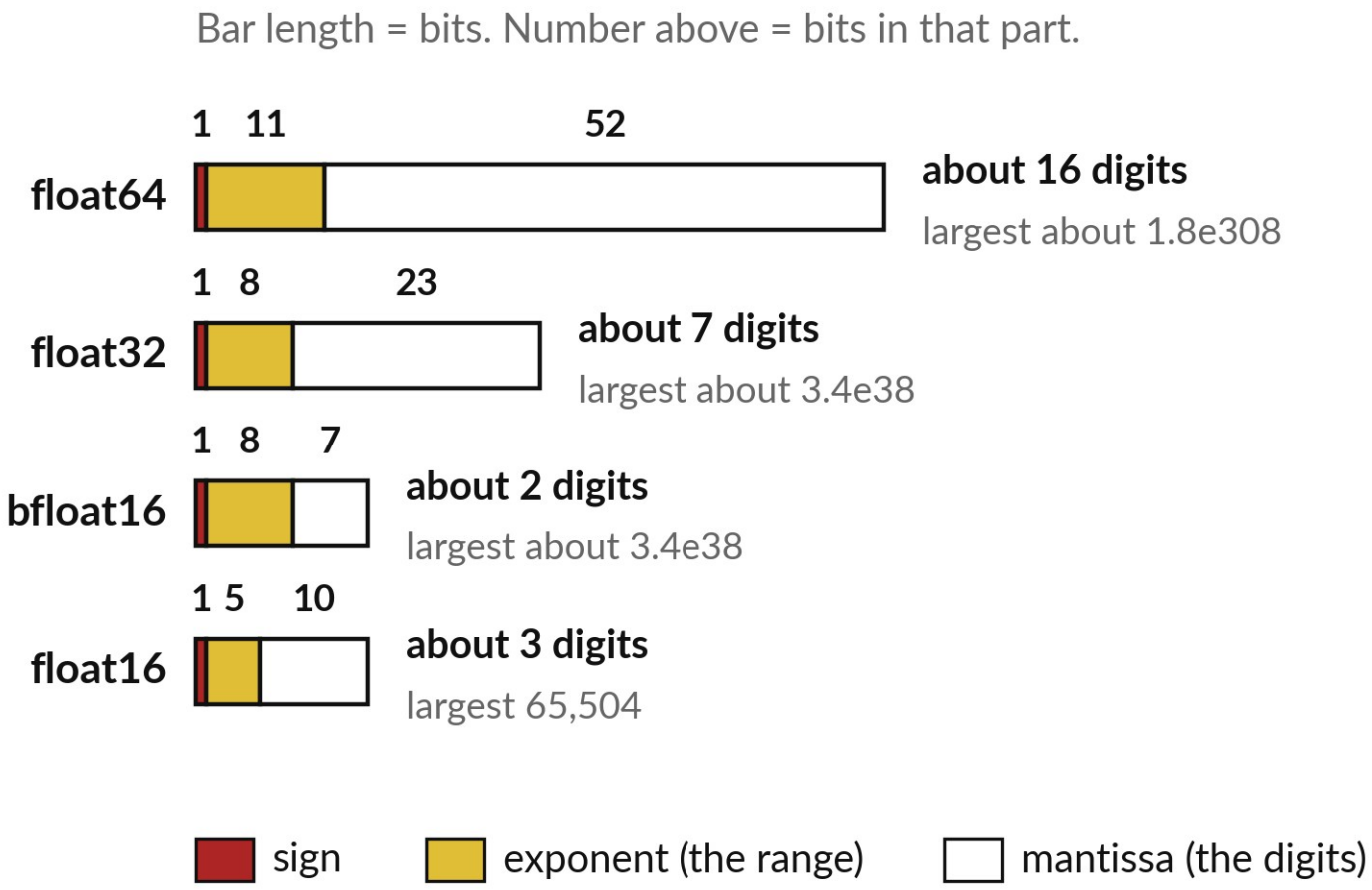
Raise e to each score (logit), then divide by the total.

softmax(z)_i = e^(z_i) / sum of e^(z_j).

Scores 2, 1, 0.1: e^z = 7.389, 2.718, 1.105; total 11.213.

Divide: 0.659, 0.242, 0.099, adding to 1.

Each number format has a largest value, and e^z passes it quickly



Fewer bits mean a smaller range and fewer digits.

e^z is inf from z = 710 (float64), 89 (float32), 12 (float16).

float16 keeps about 3 digits: 1 + 0.0004 is stored as 1.

16-bit training keeps a 32-bit weight copy and scales the loss (Micikevicius et al., 2017).

Subtracting the largest score keeps softmax exact and stops the overflow

Shift by a constant c

$$\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{z_i}\, e^{-c}}{\sum_j e^{z_j}\, e^{-c}} = \text{softmax}(z)_i$$

Symbol	What it means	In our data
z (logits)	Three large scores. The naive way: e^1000 is inf, and inf / inf is nan.	(1000, 1001, 1002): nan, nan, nan
c = the maximum	Any c cancels top and bottom. With the maximum every exponent is 0 or less.	z - 1002 = -2, -1, 0
e^(z minus c) and sum	Each is at most 1, so nothing overflows; the largest is exactly 1, so the sum is at least 1.	0.1353 + 0.3679 + 1 = 1.5032
Result	Divide each by the sum. It equals softmax(0, 1, 2): only differences matter.	0.0900, 0.2447, 0.6652

Log-sum-exp computes the log of a sum of exponentials without overflow

Log-sum-exp

$$\mathrm{LSE}(z) = \ln \sum_j e^{z_j} = m + \ln \sum_j e^{z_j - m}, \quad m = \max_j z_j$$

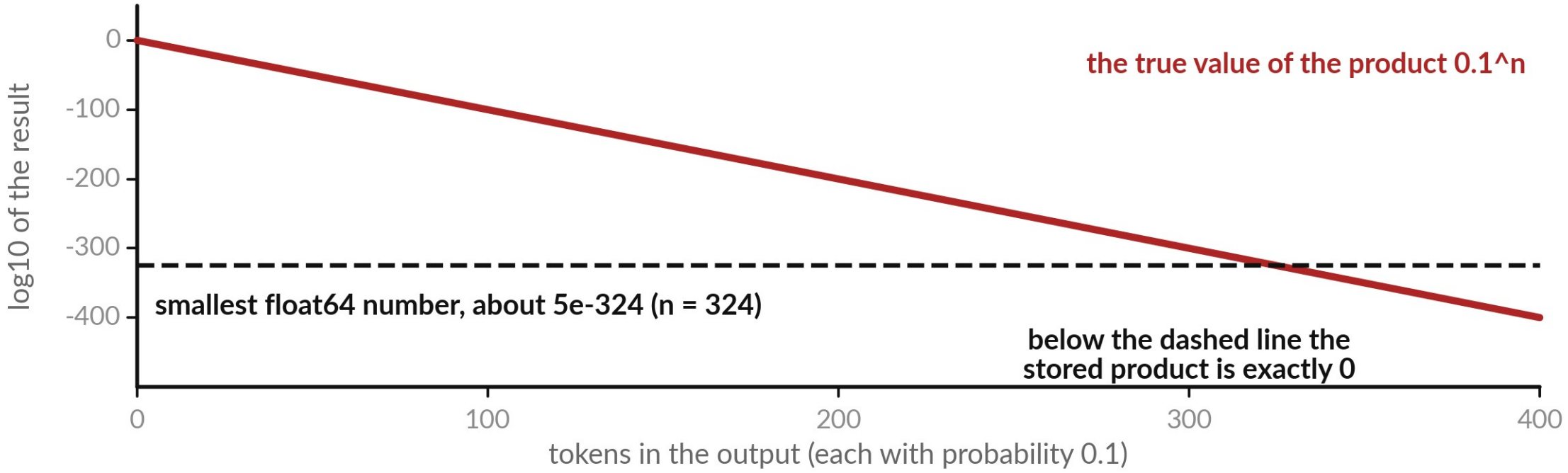
Log-softmax

$$\ln \mathrm{softmax}(z)_i = z_i - \mathrm{LSE}(z)$$

Symbol	What it means	In our data
m	The maximum of the scores. The shift keeps every exponent at most 0.	1002
LSE(z)	The log-sum-exp, in nats (natural log).	1002 + ln(1.5032) = 1002.4076
ln softmax	The log of each probability, computed directly without ever forming the probability.	-2.4076, -1.4076, -0.4076

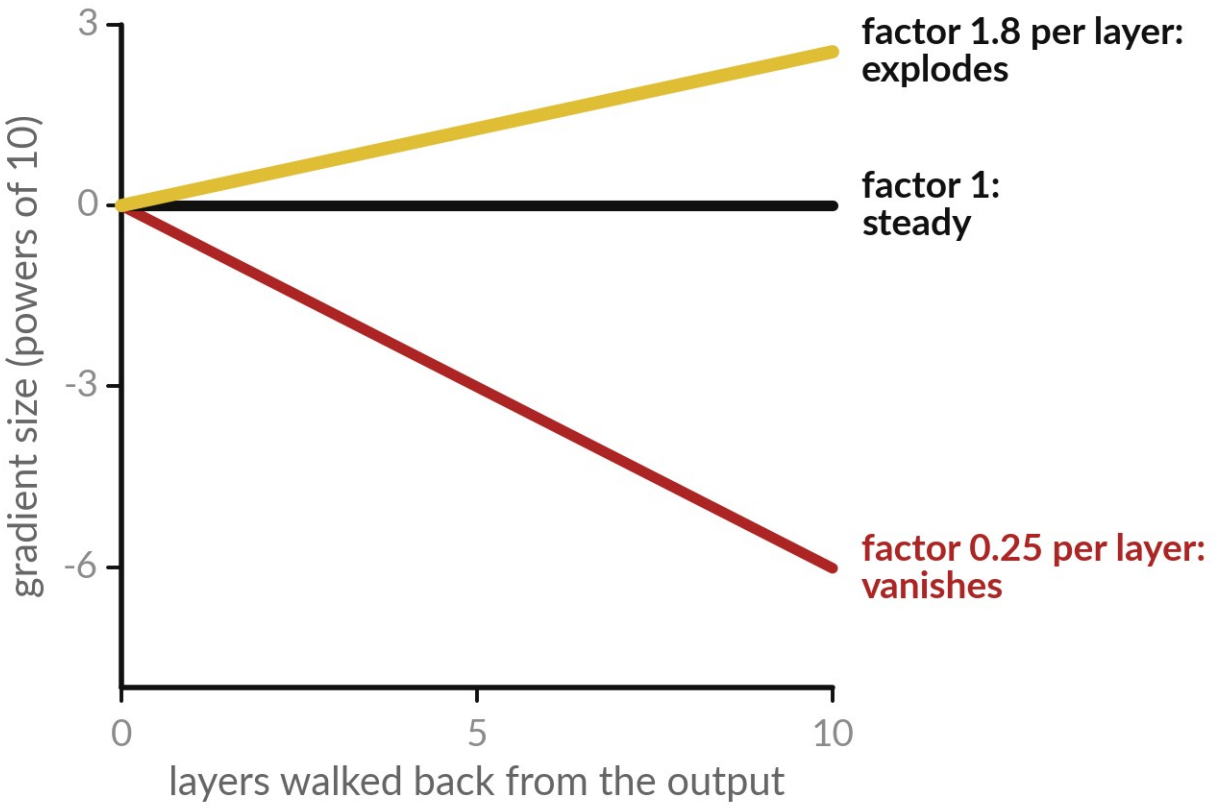
Session 1 added logs in place of multiplying probabilities. Session 17 computes cross-entropy from log-softmax.

A product of many probabilities underflows to zero, but the sum of their logs does not



Tokens of probability 0.1. The product is 0 from n = 324; the sum of logs at n = 400 is -921.03.

Vanishing and exploding gradients: a product of many factors heads to 0 or to infinity



The backward pass multiplies one factor per layer.

Factor 0.25, the largest sigmoid slope (Session 11): 0.25^10 is about 0.00000095. Vanishing.

Factor 1.8: 1.8^10 is about 357. Exploding.

Fixes: ReLU, normalisation, residual connections, clipping.

Clipping a gradient limits its length when it explodes

Clip by norm

$$g \leftarrow g \cdot \frac{c}{\|g\|_2} \quad \text{if } \|g\|_2 > c$$

Symbol	What it means	In our data
g	The gradient: the column of all the slopes.	g = (30, 40)
norm of g	The L2 norm (Session 2): the length of the vector.	sqrt(900 + 1600) = 50
c	The largest length we allow, chosen by us.	c = 5
New g	Multiply g by c / norm = 5 / 50 = 0.1. The direction stays the same.	(3, 4), length 5

Clipping changes the length of the step but not its direction. Gradient clipping is common in published training recipes.

Four practical slips, from lambda to logs, and how to spot each one

Mistake	What you see	Fix
Pick lambda by the training error	The best lambda is always 0	Pick lambda by the error on validation points
Choose lambda and report the error on the same points	A reported error that new data will not match	Keep test points apart and report their error once
Exponentiate large scores directly	inf or nan in the softmax	Subtract the maximum first, or use log-softmax
Multiply many probabilities	The result is 0 for long sequences	Add log-probabilities instead

Read the stable softmax before you write it

```
z = np.array([1000., 1001., 1002.])
m = z.max()
e = np.exp(z - m)
probs = e / e.sum()
lse = m + np.log(e.sum())
logprobs = z - lse
```

In plain words

m is the largest score. z - m is at most 0.

Every entry of e is at most 1, so it cannot overflow.

probs adds up to 1 and equals softmax(0, 1, 2).

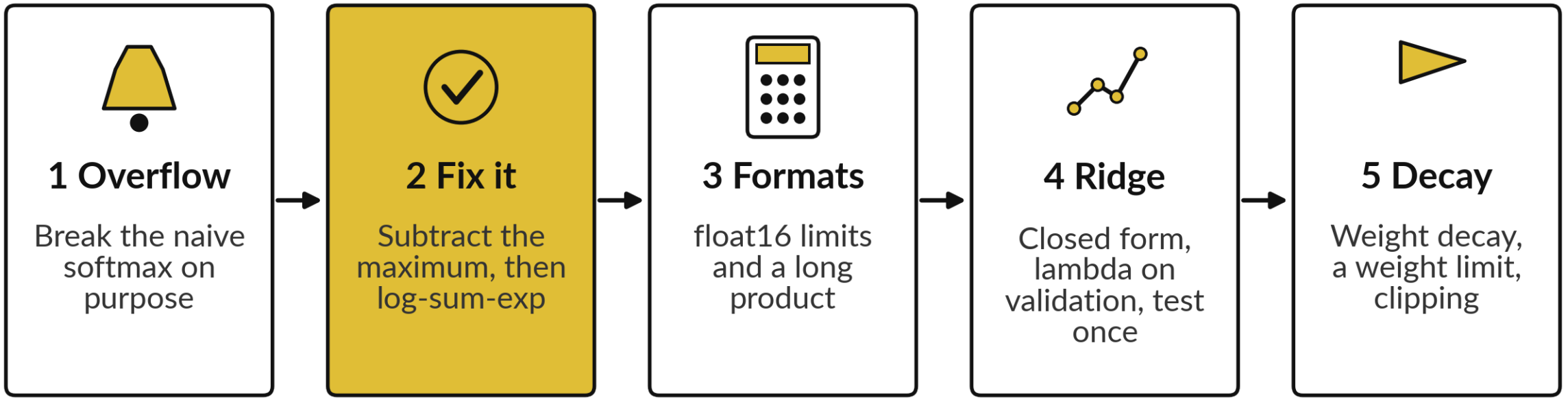
lse is the log-sum-exp. logprobs are the log of each probability, with no probability formed.



Part 2: the lab

Close the slides. Open the notebook or the workbook.

Five lab steps: break a softmax, fix it, then fit and test a ridge curve



The notebook has the same five steps; the workbook uses the sheets Softmax, Formats, Ridge and Small. Predict, run, read, change one thing.

The numbers your lab should reach, from the softmax to the test error

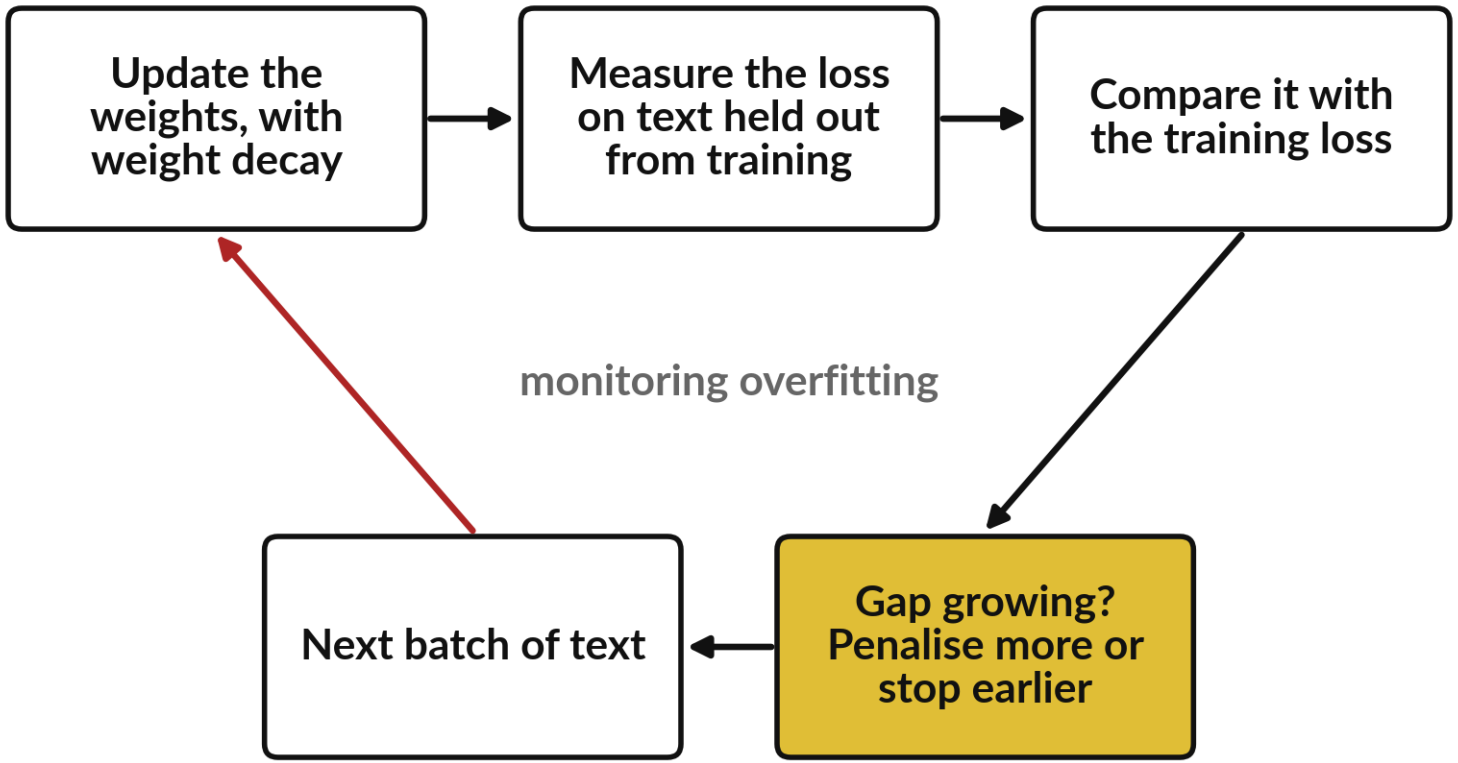
What you compute	You should see
Stable softmax of (1000, 1001, 1002)	0.0900, 0.2447, 0.6652
Log-sum-exp of (1000, 1001, 1002)	1002.4076
Largest float16 number	65504
Sum of squared weights, lambda = 0.1	1.5473
Validation error, lambda = 0 and 0.1	0.6657 and 0.1584
Test error, once, at the chosen lambda = 0.1	0.1730
Weight after one step with decay (w = 2)	1.7

If your number differs, find the step that went wrong. Do not change a number to match.

Warnings and errors in this lab, and which of them are on purpose

What you see	What it means	What to do
RuntimeWarning: overflow in exp, and nan	The naive softmax overflowed. This is expected in step 1	Continue to step 2 and subtract the maximum
Huge or jumpy weights at lambda = 0	X^T X is badly conditioned, so the answer is unreliable	Use lambda > 0 in the closed form
Weights look huge	lambda is too small	Raise lambda and compare the norm
#NUM! in a sheet	EXP of a large number overflowed	Subtract the maximum first

Where this appears in AI: how overfitting is watched and controlled

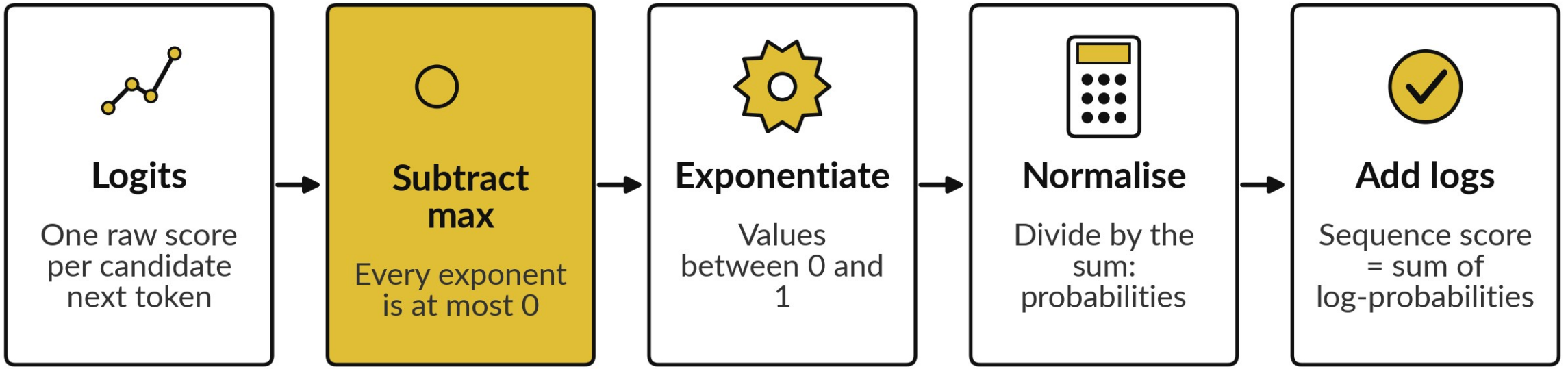


The loss on held-out text tells us whether the model memorises.

Published work describes weight decay, dropout (randomly switching off units while training) and early stopping (stop when the held-out loss rises).

Closed products do not publish their settings.

Where this appears in AI: why a very long answer does not overflow



Published practice for models of this kind: softmax computed in a stable way, and sequence scores kept as sums of logs.

Check an AI assistant with your own numbers

10 minutes Hands-on, with no private data

Ask any AI assistant to compute a softmax that overflows. Then compare with your table.

Type: "Compute softmax of the scores 1000, 1001 and 1002 without overflow. Show every step."

Expected result: 0.0900, 0.2447, 0.6652.

Did it subtract the maximum? If its answer differs, find the first number that differs from yours.

Never paste confidential data.

You keep

A note: did the assistant match your table?

Exit check: three questions with short answers

Question	Short answer
What does the L2 penalty do to the weights, and which points choose lambda?	It shrinks the weights towards zero. Validation points choose lambda; test points are used once, afterwards.
Why does subtracting the maximum not change the softmax?	The factor e^(-c) appears in the top and in every term of the bottom, so it cancels. It only keeps the exponents at most 0.
Why are log-probabilities added instead of probabilities multiplied?	A product of many probabilities underflows to 0. The sum of their logs is an ordinary number.

Save your lab and keep the two rules

What you keep from today

Keep your saved notebook or workbook, with the ridge fit for several values of lambda, the test error, and the stable softmax. Write down the rule: subtract the maximum before you exponentiate.

Our question today

How do we limit memorising in a model with billions of numbers, and why does a very long answer not overflow the computer?

Next session

Session 13: probability basics: random variables, expectation and variance.

Take away

Penalty
lambda × sum of w^2

Chosen lambda
0.1, test error 0.173

Stable softmax
subtract the maximum

Next session
Probability basics